

Q Search

Get started

Overview

Quick start

Model list

Pricing

Region availability

Model invocation

Text generation

Multi-modal understanding

Video generation

Image generation

Embeddings

Tool use

Advanced usage

Deep reasoning

Context management

Streaming output

Prefill-based response

Context caching

Files API tutorial

Visual grounding

Prompt engineering

Structured output (beta)

Integrate with third-party tools

Home > ModelArk > Model invocation > Advanced usage > Deep reasoning

Copy Page

Deep reasoning

Глубокое понимание означает, что модель анализирует проблему и выполняет многоэтапное планирование, прежде чем дать ответ, а затем пытается решить проблему. Это хорошо подходит для сложных и абстрактных сценариев, таких как программирование, научные рассуждения и рабочие процессы с участием агентов. После включения функции глубокого понимания в указанном параметре возвращается цепочка рассуждений, которую можно использовать для наблюдения и анализа.

Совет

Если вы новичок в ModelArk, ознакомьтесь с [кратким руководством](#), чтобы быстро приступить к работе.

Быстрый старт

Примеры кода

< Curl Python Go Java OpenAI SDK >

```
Bash
1 curl https://ark.ap-
  southeast.bytepluses.com/api/v3/chat/completions
  \
2 -H "Тип содержимого: application/json" \ "Тип
  содержимого: application/json" \
3 -H "Авторизация: на предъявителя $ARK_API_KEY"
  \ "Авторизация: на предъявителя $ARK_API_KEY" \
4 -d '{
5   "модель": "seed-2-0-lite-260228",
6   "сообщения": [
7     {
8       "роль": "пользователь",
9       "контент": "Я хочу изучить различия между
      моделями глубокого рассуждения и моделями без
      глубокого рассуждения. Как мне
      продемонстрировать свою компетентность?"
```

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding ▼
- Video generation ▼
- Image generation ▼
- Embeddings ▼
- Tool use ▼
- Advanced usage ^

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching ▼
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)

Integrate with third-party tools

S

☰

ри необходимости замените идентификатор модели. Чтобы запросить идентификатор модели, см. [Список моделей](#).

Модели и API

Поддерживаемые модели: [Deep reasoning](#).

Поддерживаемые API:

- **Responses API:** недавно запущенный API с упрощенным управлением контекстом, расширенными возможностями вызова инструментов и кэшированием для снижения затрат. Рекомендуется для новых компаний и пользователей.
- **Chat API:** широко используемый API с низкими затратами на миграцию для существующих компаний.

Базовое использование

Многоходовой диалог

Комбинируя системные сообщения, сообщения модели и сообщения пользователя, вы можете реализовать многоходовой диалог. Если вам нужно продолжить диалог в рамках одной темы, вы можете передать модели историю предыдущих диалогов.

Метод в...	Управление контекстом вручную
Пример	JSON 1 ... 2 "модель": "начальная версия-2-0-lite-260228", "модель": "начальная версия-2-0-lite-260228", 3 "сообщения": ["сообщения": [4 { "роль": "пользователь", "контент": "Привет, расскажи анекдот". }, { "роль": "пользователь", "контент": "Привет, расскажи анекдот". }, 5 { "role": "assistant", "content": "Почему учебник по математике выглядел грустным? Потому что в нем было слишком много задач! 😊" }, { "role": "assistant", "content": "Почему учебник по математике выглядел грустным? Потому что в нем было слишком много задач! 😊" }, 6 { "role": "user", "content": "В чем соль этой шутки?" } { "role": "user", "content":

Q Search

⌕ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding ▼
- Video generation ▼
- Image generation ▼
- Embeddings ▼
- Tool use ▼
- Advanced usage ^

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching ▼
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

S

≡

Chat API

При создании контекста для многоэтапного диалога:

- Версии модели до 251228 : удалите параметр **reasoning_content** из истории диалога и оставьте только **role** и **content** . ModelArk попытается проигнорировать этот параметр, но его явное удаление гарантирует корректность структуры запроса.
- seed-1.8 и более поздние модели: оставьте параметр **reasoning_content** в истории диалога. Модель определяет, следует ли включать этот параметр во входные данные для логического вывода.

Дополнительные сведения и полные примеры см. в разделе [Управление контекстом](#).

Потоковая передача вывода

Контент выводится динамически по мере его генерации большой моделью. Вы можете видеть промежуточные результаты в процессе, не дожидаясь завершения логического вывода модели.

Demo	Advantages
	• Improved waiting experience : You can process in-progre content immediately without waiting for the full content to generated. • Real-time feedback : In multi-turn interactions, you can tra the current processing stage of the task in real time. • Higher fault tolerance : If an error occurs midway, you car obtain the generated content, avoiding the case where nor streaming output fails and returns nothing. • Simplified timeout management : Keeps the client-server connection active and reduces timeouts for long-running c complex tasks.

Enable streaming output by setting **stream** to `true` .

JSON

```
1      "model": "seed-2-0-lite-260228",
2      "messages": [
3        {"role": "user", "content": "Differences
between deep reasoning models and non-deep
reasoning models"}
4      ],
5      "stream": true
```

Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

le/disable deep reasoning

The `thinking` parameter controls whether to turn off deep reasoning, enabling fine-grained control: "use deep reasoning for complex tasks and respond efficiently to simple tasks", to reduce costs and improve efficiency.

- Valid values:
 - `enabled` : Force enable. Forcibly turn on the deep reasoning capability.
 - `disabled` : Force disable. Forcibly turn off the deep reasoning capability.
- Code samples:

< Curl Python Go Java OpenAI SDK >

```
Bash
1  curl https://ark.ap-
    southeast.byteplus.com/api/v3/chat/completi
    ns \
2      -H "Content-Type: application/json" \
3      -H "Authorization: Bearer $ARK_API_KEY"
    \
4      -d '{
5          "model": "seed-2-0-lite-260228",
6          "messages": [
7              {
8                  "role": "user",
9                  "content": [
10                     {
11                         "type":"text",
12                         "text":"I want to
research the differences between deep
reasoning models and non-deep reasoning
models, and demonstrate my professionalism"
13                     }
14                 ]
15             },
16             ],
17             "thinking":{
18                 "type":"disabled"
19             }
20         }'
```

- `model`: Replace it with the model actually called.
- `thinking.type`: valid values:
 - `disabled` : Force disables deep reasoning. The model does not output chain-of-thought content.
 - `enabled` : Force enables deep reasoning. The model is forced to output chain-of-thought content.

Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding ▼
- Video generation ▼
- Image generation ▼
- Embeddings ▼
- Tool use ▼
- Advanced usage ^

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching ▼
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

S

≡

disabled .

deepseek-v4-pro-260425: supports enabled (default) and disabled .

- deepseek-v4-flash-260425: supports enabled (default) and disabled .
- seed-2-0-lite-260428: supports enabled (default) and disabled .
- seed-2-0-mini-260428: supports enabled (default) and disabled .
- seed-2-0-pro-260328: supports enabled (default) and disabled .
- seed-2-0-lite-260228: supports enabled (default) and disabled .
- seed-2-0-mini-260215: supports enabled (default) and disabled .
- seed-2-0-code-preview-260328: supports enabled (default) and disabled .
- seed-1-8-251228: supports enabled (default) and disabled .
- seed-1-6-250915: supports enabled (default) and disabled .
- seed-1-6-250615: supports enabled (default) and disabled .
- seed-1-6-flash-250715: supports enabled (default) and disabled .
- seed-1-6-flash-250615: supports enabled (default) and disabled .
- gpt-oss-120b-250805: supports enabled (default), disabled , and auto .
- glm-4-7-251222: supports enabled (default) and disabled .
- deepseek-v3-2-251201: supports enabled and disabled (default).

- More information
 - For instructions on using the Responses API, see [Control deep reasoning](#).
 - Deep reasoning affects prefill-based response. For details, see [Prefill-based response](#).

Set maximum output length

The model output consists of two parts: **Chain of Thought (COT)** and **final answer (Answer)** . Control the model output length properly to balance quality, speed, cost, and stability.

Input m...	Manage context manually
API	Chat API
Example	JSON <pre>1 ... 2 "model": "seed-2-0-lite-260228", 3 "messages": [4 {"role": "user", "content": "Hi, tell a joke."}</pre>

Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding ▼
- Video generation ▼
- Image generation ▼
- Embeddings ▼
- Tool use ▼
- Advanced usage ^

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching ▼
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)

Integrate with third-party tools

For complete examples and more information, see [Control output \(answer + Chain of Thought\) length](#).

Adjust chain-of-thought length

Use the **reasoning_effort** (Chat API) and **reasoning.effort** (Responses API) parameters to adjust the chain-of-thought length, balancing quality, latency, and cost requirements across scenarios. Valid values are:

- `none` : Turn off reasoning. This option only works for specified models. It will not work if passed for unsupported models. Supported model: `glm-5-2-260617` .
- `minimal` : Turns off reasoning and answers directly.
- `low` : Light reasoning, focused on fast responses.
- `medium` (default): Balanced mode, considering both speed and depth.
- `high` : Deep analysis for handling complex problems.
- `xhigh` : Higher level of reasoning. This option only works for specified models. It will not work if passed for unsupported models. Supported model: `glm-5-2-260617` .
- `max` : Maximum level of reasoning, suitable for highly difficult reasoning tasks. This option only works for specified models. It will not work if passed for unsupported models. Supported models for this option:
 - `glm-5-2-260617`
 - `deepseek-v4-pro-260425` (currently supports only Chat API; Responses API support is pending)
 - `deepseek-v4-flash-260425` (currently supports only Chat API; Responses API support is pending)

Note

The default value for **reasoning_effort** (Chat API) and **reasoning.effort** (Responses API) is `medium` .

- The default value of the following models is `high` :
 - `dola-seed-2-1-turbo-260628`
- The default value of `glm-5-2-260617` is `max` . To ensure compatibility, some values for this model are processed as follows:
 - If `minimal` is passed: reasoning is turned off;
 - If `low` / `medium` is passed: it is processed as equivalent to `high` ;
 - If `xhigh` is passed: it is processed as equivalent to `max` .

Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)

Integrate with third-party tools

s

```
2      "model": "seed-2-0-lite-260228",
3      "messages": [
4        {
5          "role": "user",
6          "content": "What are some common cruciferous plants?"
7        },
8      ],
9      "reasoning_effort": "low"
10    }
```

```
2      "model": "seed-2-0-lite-260228",
3      "input": "What are some common cruciferous plants?"
4    },
5    ],
6    "reasoning_effort": "low"
7  }
```

Supported models

- deepseek-v4-pro-260425
- dola-seed-2-1-turbo-260628
- deepseek-v4-flash-260425
- seed-2-0-lite-260428
- seed-2-0-mini-260428
- seed-2-0-pro-260328
- seed-2-0-lite-260228
- seed-2-0-mini-260215
- seed-2-0-code-preview-260328
- seed-1-8-251228

For the complete example and instructions, see [Control CoT length \[New\]](#).

Output thinking summary

Supported models

- dola-seed-2-1-turbo-260628
- seed-2-0-lite-260428
- seed-2-0-mini-260428
- seed-2-0-pro-260328

Instructions

The thinking summary is enabled by default. When it is enabled, the summary of the model's thinking content (`choices.message.reasoning_content`) and the encrypted thinking content (`choices.message.encrypted_content`) will be returned instead of the thinking content.

Tip

Get started

Overview

Quick start

Model list

Pricing

Region availability

Model invocation

Text generation

Multi-modal understanding

Video generation

Image generation

Embeddings

Tool use

Advanced usage

Deep reasoning

Context management

Streaming output

Prefill-based response

Context caching

Files API tutorial

Visual grounding

Prompt engineering

Structured output (beta)

Integrate with third-party tools

S



eter explanation:

- **reasoning_effort** : Only applies to the original thinking content of the model, not to the thinking summary.
- **usage.completion_tokens_details.reasoning_tokens** : Tokens of the original thinking content. Billing is still calculated based on the tokens of the original thinking content.

Pass thinking content back

If you need to pass the thinking content back when calling tools, note the following:(For an example of passing thinking content back, see [Pass encrypted thinking content back](#).)

- Pass both **encrypted_content** and **reasoning_content** : **encrypted_content** has higher priority, and the content in **reasoning_content** will be ignored. The content of **encrypted_content** must be valid. If it is tampered with, it cannot be restored.
- Only pass **reasoning_content** : The thinking content summary is used for inference. In multi-turn tool call scenarios (such as agent scenarios), if **encrypted_content** is not passed back, the model inference quality will be degraded.
- No parameters related to thinking content are passed back: No error will be reported.

Tool calling

Models earlier than seed-1.8 will directly discard chain-of-thought content if deep thinking is enabled when tools are called. To provide more detailed and accurate responses, seed-1.8 and some other models will not directly discard chain-of-thought content. The chain-of-thought content may participate in subsequent rounds of reasoning, which will increase input tokens. For details, see [How deep reasoning works](#).

Tip

It is recommended to use `previous_response_id` in Responses API, and then ModelArk automatically saves the historical conversation context and passes it back to the inference service during multi-turn interactions.

Pass encrypted thinking content back

Thinking summary is enabled by default for seed-2-0-pro-260328, seed-2-0-mini-260428, seed-2-0-lite-260428 and later versions, so they will not output the original thinking content. The following is an example of passing the encrypted thinking content back when tools are called. When using Responses API, it is recommended to use `previous_responses_id` to automatically obtain the original thinking content and pass it back to the model for inference.

8 K

Overview

Overview

Quick start

Model list

Pricing

Region availability

Text generation

Text generation

Multi-modal understanding Video generation

Image generation

Embeddings

Tool use

Advanced usage

Context management

Context management

Streaming output

Prefill-based response

Context caching

Files API tutorial

Visual grounding

Prompt engineering

Structured output (beta)

Integrate with third-party tools

- Pass thinking content back

explanatio
ns

Request of round 1: Trigger tool calling

Bash

```
1 curl https://ark.ap-
southeast.bytepluses.com/api
/v3/chat/completions \
2 -H "Content-Type:
application/json" \
3 -H "Authorization: Bearer
$ARK_API_KEY" \
4 -d '{
5     "model": "seed-2-0-lite-
260228",
6     "messages": [
7         {
8             "role":
"system",
9
10            "content": "You
are an AI assistant."
11        },
12        {
13            "role": "user",
14
15            "content":
"What is the weather like
in Beijing today?"
16        }
17    ],
18    "thinking":{"type":
"enabled"},
19    "tools": [
20        {
21            "type":
"function",
22            "function": {
23                "name":
"get_weather",
24
25            "description": "Weather
query",
26
27            "parameters": {
28                "properties": {
```

Request c

Bash

1	c
2	/
3	B
4	a
5	l
6	
7	
8	"
9	
10	"
11	
12	
13	"
14	
15	"
16	i
17	
18	"
19	[
20	t
21	
22	"
23	
24	
25	"
26	q

Q Search

K

Get started

Overview

Quick start

Model list

Pricing

Region availability

Model invocation

Text generation

Multi-modal understanding

Video generation

Image generation

Embeddings

Tool use

Advanced usage

Deep reasoning

Context management

Streaming output

Prefill-based response

Context caching

Files API tutorial

Visual grounding

Prompt engineering

Structured output (beta)

Integrate with third-party tools

```
30
    "description": "The
    location for which to query
    the weather.",
31
    "type": "string"
32      },
33    },
34
    "required": [
35      "location"
36    ],
37    "type":
    "object"
38      },
39    }
40  }
41 ]
42 }
```

Response of round 1: Return the tool call instruction

The model returns key parameters such as `encrypted_content` , `reasoning_content` , and `tool_calls` .

Bash

```
1 {
2   "choices": [
3     {
4       "finish_reason":
       "tool_calls",
5       "index": 0,
6       "logprobs":
       null,
7       "message": {
8         "content":
9         "",
10
11       "reasoning_content": "I
will call the relevant
tools to query the weather
in Beijing.\n",
12
13       "encrypted_content":
"djF+2EICEj3ryfEfSUdR/SmS80e
EH4zn0YftL4SWDXR8uxR0jx11W7r
RCj5ArLwzsm7rFs04frOdLm2p3/y
Wz/r0TMqrjHiaTTvRMNdV6sLdETy
```

```
--
"
29
"
30
"
31
32
"
33
l
t
34
35
"
36
37
38
39
```

Response instruction

The inform key param argument:

Bash

```
1 {
2
3
1
"
f
4
4
3
5
2
6
7
8
9
"
0
c
10
11
12
13
"
14
```

Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

```
11  i5R7M2QrSYmIUJjnoB48IVUCzu4x
12  n7bT05qheVnG09fbs15gYK3zINUv
13  Vsp510q72U/ksrPZFVs2BTgNRwjm
14  xnFNn7A==" ,
15  "role":
16  "assistant",
17
18  "tool_calls": [
19
20    {
21      "function": {
22        "arguments": "
23        {\\"location\\":
24        \\"Beijing\\"}",
25        "name": "get_weather"
26      },
27      "id":
28      "call_wiezxeyae8jzx13jx8nhfg
29      b5",
30      "type": "function"
31    }
32  ]
33 }
```

Request of round 2: Pass back the full context and generate the final response

The following example shows the following process: based on the request of round 1, pass back the encrypted original thinking content (**encrypted_content**), thinking content summary (**reasoning_content**) and tool calling results, then the model generates a response in natural language.

```
Bash
1  curl https://ark.ap-
2    southeast.bytepluses.com/api
3    /v3/chat/completions \
4    -H "Content-Type:
5    application/json" \
6    -H "Authorization: Bearer
7    $ARK_API_KEY" \
```

Request generate

Pass in int
such as th
then the n
language.

BytePlus

Q Search

⌘ K

Get started

Overview

Quick start

Model list

Pricing

Region availability

Model invocation

Text generation

Multi-modal understanding

Video generation

Image generation

Embeddings

Tool use

Advanced usage

Deep reasoning

Context management

Streaming output

Prefill-based response

Context caching

Files API tutorial

Visual grounding

Prompt engineering

Structured output (beta)

Integrate with third-party tools

S

⋮

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

----- ,

"messages": [

{

"role":

"system",

"content": "You

are an AI assistant."

},

{

"role": "user",

"content":

"What is the weather like

in Beijing today?"

},

{

"reasoning_content": "<the

reasoning content from the

previous request>"

"encrypted_content":

"djF+2EICEj3ryfEfSUdR/SmS80e

EH4zn0YftL4SWDXR8uxROjx11W7r

RCj5ArLwzsm7rFs04frOdLm2p3/y

Wz/r0TMqrjHiaTTvRMNdV6sLdETy

Slb3PDgY1W+zuYuETiq3bQuxga5j

Kx+Gpfv1DJMfJfzq/G1kDp6ryurs

0rKAFIziyc4mfFSh2CzDKNcAcp5F

i5R7M2QrSYmIUJjnoB48IVUCzu4x

n7bT05qheVnG09fbs15gYK3zINUv

Vsp510q72U/ksrPZFVs2BTgNRwjm

xnFNn7A=="

"role":

"assistant",

"tool_calls": [

{

"function": {

"arguments": "

{"location\":

\"Beijing\"}"

"name": "get_weather"

},

"id":

"call_wiezxeyae8jzx13jx8nhfg

b5",

"type":

"function"

S

/

2

B

3

a

4

5

l

6

7

8

"

9

"

7

10

11

"

12

13

14

"

"

f

4

15

"

16

17

18

"

19

"

20

21

"

q

22

"

23

"

24

"

25

"

26

27

28

"

L

t

29

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)

Integrate with third-party tools

```
34         {
35             "role": "tool",
36             "tool_call_id": "call_wiezxey
37             ae8jzx13jx8nhf gb5",
38             "content": "5
39             degree"
40         },
41         "thinking": {"type":
42             "enabled"},
43         "tools": [
44             {
45                 "type":
46                 "function",
47                 "function": {
48                     "name":
49                     "get_weather",
50
51                     "description": "Weather
52                     query",
53
54                     "parameters": {
55                         "properties": {
56                             "location": {
57
58                             "description": "The
59                             location for which to query
60                             the weather.",
61
62                             "type": "string"
63                         }
64                     },
65                     "required": [
66                         "location"
67                     ],
68                     "type":
69                     "object"
70                 }
71             }
72         ]
73     }'
74 }
```

32

33

34

35



Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)

Integrate with third-party tools

S

≡

it by default. The following is an example of passing the original g content back when tools are called.

API	Chat API
Supported models	seed-2-0-lite-260428 seed-2-0-mini-260428 seed-2-0-pro-260328 seed-2-0-lite-260228 seed-2-0-mini-260215 seed-1-8-251228 deepseek-v3-2-251201

Example Request of round 1: Trigger tool calling

Bash

```
1 curl https://ark.ap-
southeast.bytepluses.com/api/v3/chat/comp:
etions \
2   -H "Content-Type: application/json" \
3   -H "Authorization: Bearer $ARK_API_KEY"
\
4   -d '{
5     "model": "seed-2-0-lite-260228",
6     "messages": [
7       {
8         "role": "system",
9
10        "content": "You are an AI
assistant."
11      },
12      {
13        "role": "user",
14
15        "content": "What is the
weather like in Beijing today?"
16      }
17    ],
18    "thinking":{"type": "enabled"},
19    "tools": [
20      {
21        "type": "function",
22        "function": {
23          "name": "get_weather",
24
25          "description": "Weather
query",
26
27          "parameters": {
28            "properties": {
"location": {
```

Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

```
query the weather.",
31                                     "type":
32                                     "string"
33                                     },
34                                     "required": [
35                                     "location"
36                                     ],
37                                     "type": "object"
38                                     }
39                                     }
40                                     }
41                                     ]
42                                     }'
```

Response of round 1: Return the tool call instruction

The model returns key parameters such as `reasoning_content` `tool_calls` .

```
JSON
1  {
2      "choices": [
3          {
4              "finish_reason": "tool_calls",
5              "index": 0,
6              "logprobs": null,
7              "message": {
8                  "content": "",
9
10                 "reasoning_content": "The
11                 user wants to check today's weather in
12                 Beijing, which matches the functionality
13                 of the get_weather tool. The location
14                 parameter needs to be set to Beijing.
15                 Therefore, I will call this tool to
16                 retrieve the weather information."
17                 "role": "assistant",
18                 "tool_calls": [
19                     {
20                         "function": {
21                             "arguments":
22                             " {\"location\": \"Beijing\"}",
23                             "name":
24                             "get_weather"
25                         },
26                         "id":
27                         "call_wiezkeyae8jzx13jx8nhfgb5",
28                         "type": "function"
29                     }
30                 ]
31             }
32         }
33     ]
34 }
```

Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

```
27 }
```

Request of round 2: Pass back the full context and generate final response

Based on the request of round 1, pass back the chain-of-thou content and tool calling results, then the model generates a response in natural language.

Bash

```
1 curl https://ark.ap-
southeast.bytepluses.com/api/v3/chat/completions \
2   -H "Content-Type: application/json" \
3   -H "Authorization: Bearer $ARK_API_KEY" \
4   -d '{
5     "model": "seed-2-0-lite-260228",
6     "messages": [
7       {
8         "role": "system",
9
10        "content": "You are an AI
assistant."
11      },
12      {
13        "role": "user",
14
15        "content": "What is the
weather like in Beijing today?"
16      },
17      {
18
19        "reasoning_content": "<the
reasoning content from the previous
request>"
20      },
21      {
22        "role": "assistant",
23        "tool_calls": [
24          {
25            "function": {
26              "arguments": "
{\"location\": \"Beijing\"}",
27              "name":
28              "get_weather"
29            },
30            "id":
31            "call_wiezkeyae8jzx13jx8nhfgb5",
32            "type": "function"
33          }
34        ]
35      }
36    ]
37  }
```

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

```
gb5",
36
37         "content": "5 degree"
38     }
39 ],
40 "thinking":{"type": "enabled"},
41 "tools": [
42     {
43         "type": "function",
44         "function": {
45             "name": "get_weather",
46
47             "description": "Weather
query",
48             "parameters": {
49                 "properties": {
50                     "location": {
51
52             "description": "The location for which to
query the weather.",
53                 "type":
"string"
54             }
55         },
56         "required": [
57             "location"
58         ],
59         "type": "object"
60     }
61 }
62 }
63 ]
64 }'
```

Instructions

How it works

- Multi-turn conversations

Flowchart	Description
	During each turn of the conversation, the deep

Q Search

⌕ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

chain-of-thought (CoT) content and the final answer.

- In the next conversation turn, the previously output chain-of-thought content will not be spliced into the context. The chain-of-thought content shows the model's problem-solving process, including splitting the problem into multiple sub-problems to process, generating multiple responses and synthesizing a better answer, etc.

- Tool use (seed-1.8 and later models)

After enabling deep reasoning in the tool use scenario, the chain-of-thought content will not be discarded directly to provide more detailed and accurate answers. The chain-of-thought content from historical turns will participate in inference as needed (judged independently by the model). During the entire request process, users only need to send back the complete context. The server will independently judge whether to retain the chain-of-thought content. The chain-of-thought content that is not input to the model will not be counted in the token usage. For code samples, see [Tool calling](#).

Flowchart	Description
	When answering

Q Search

⌕ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

1.2), the model gives the answer after multiple thinking processes and tool calls. ModelArk will input the complete context including the chain-of-thought content to the model for processing.

- When starting to answer question 2 (request 2.1), ModelArk will independently judge and delete the chain-of-thought in the previous context before inputting it to the model.

Reduce request timeout failures

Deep reasoning models output chain-of-thought content, resulting in longer responses and slower speed. Therefore, tasks are very likely to fail due to timeout. Especially in non-streaming output mode, if the connection is disconnected before the task is complete, no content is output, but token usage fees are still incurred.

You can use streaming output or set a longer timeout to reduce timeout failures:

- Use streaming output (recommended): Generated content is returned immediately in chunks, which can effectively keep the connection active (avoiding connection interruptions caused by no response for a long time). This is an efficient and reliable output method (for code samples and instructions, see [Streaming output](#)). If the current application uses non-streaming output, you can change it to: use the streaming API to

Q Search

⌕ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)
- Integrate with third-party tools

...about failures.

S **≡** rease the timeout parameter: In non-streaming output scenarios, we recommend setting the `timeout` parameter to more than 30 minutes and further adjusting the timeout based on the probability of timeout triggers. Also note the TCP Keep-Alive settings in the network link (the `tcp_keepalive_time` parameter) to avoid connection interruptions by the system, firewall, router, and so on due to no data transmission for a long time.

Special note for the ModelArk Go SDK : Whether or not streaming output is used, you must set the SDK timeout parameter to more than 30 minutes.

Use batch inference for higher throughput

When your business needs to process large amounts of data and does not require timely model responses, you can use batch inference to obtain a quota of at least 10B tokens/day and reduce batch inference costs. Batch inference supports both task-based and Chat-like API call methods. To use batch inference, see [Batch inference](#) for details.

Prompt engineering techniques

Deep reasoning models analyze and break down problems on their own (chain of thought), so the focus of prompts differs from that of ordinary models.

- In addition to the problem to be solved, the prompt should include more information such as goals and scenarios. For example, specify requirements such as using English or languages like Python; information about the target audience, such as elementary school students or leaders; scenario information such as writing a paper, completing a project report, or writing a script; and goal information such as demonstrating my professionalism or gaining leadership recognition.
- Reduce or avoid descriptions that break down the problem, such as thinking step by step or using examples, because these will constrain the model's reasoning logic.
- Reduce the use of system prompts. Ask all prompt information directly through the user prompt (`role: user`).

Последнее обновление: 16 июля 2026 г. в 20:56:47

Была ли эта страница полезной для вас?

👍 Yes

👎 No

Q Search

⌘ K

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding
- Video generation
- Image generation
- Embeddings
- Tool use
- Advanced usage

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)

Integrate with third-party tools

Q Search

⌕ K

ferences

Get started

- Overview
- Quick start
- Model list
- Pricing
- Region availability

Model invocation

- Text generation
- Multi-modal understanding ▾
- Video generation ▾
- Image generation ▾
- Embeddings ▾
- Tool use ▾
- Advanced usage ▴

Deep reasoning

- Context management
- Streaming output
- Prefill-based response
- Context caching ▾
- Files API tutorial
- Visual grounding
- Prompt engineering
- Structured output (beta)

Integrate with third-party tools

Корпоративный партнер в области искусственного интеллекта от

